

SLMFix: Leveraging Small Language Models for Domain Specific Language Error Fixing with Reinforcement Learning

Anonymous ACL submission

Abstract

Large language models (LLMs) have shown impressive capabilities in code generation across many programming languages but even state-of-the-art LLMs generate programs that contain syntactic errors and fail to complete the given tasks, especially for low-resource programming languages (LRPLs). In addition, the high cost of training makes finetuning LLMs unaffordable for those with constrained computational resources, further weakening the effectiveness of LLMs for code generation. In this work, we propose **SLMFix**, a novel code generation pipeline that leverages a small language model (SLM) finetuned using reinforcement learning (RL) techniques to fix syntactic errors in LLM-generated programs for domain-specific languages (DSLs) based on interpreter feedback. Our experimental results demonstrate the effectiveness and generalizability of our approach across multiple DSLs, improving the validator pass rates by 40% on LRPLs and eliminating more than 50% of syntactic errors for high-resource DSLs. Notably, SLMFix brings substantial performance improvement to the base model and outperforms supervised finetuning approach even for 7B models on LRPLs including Ansible and Lean, showing the potential of our approach in improving the quality of LLM-generated programs.

1 Introduction

Program synthesis, a process that automatically generates a program from some (hopefully higher-level) description, has attracted the attention of many researchers since almost the beginning of the modern study of computer science (Manna and Waldinger, 1971; Waldinger and Manna, 1975; Manna and Waldinger, 1979). As transformer-based large language models (LLMs) have shown strong performance in reasoning-intensive tasks, researchers have explored the possibility of leveraging LLMs for code generation, and recent state-of-the-art models including GPT-5 (OpenAI, 2026)

and Gemini (Gemini Team, 2025) have achieved performance comparable to top-tier human programmers (Li et al., 2022; OpenAI et al., 2025), while agentic systems like GPT Codex (Chen et al., 2021) and Claude (Anthropic, 2024) have seen wide adoption in industry.

Despite their success in code generation tasks across many programming languages, the current training and inference pipeline that leverages supervised finetuning techniques for direct code generation still faces several critical challenges. First, unlike a typical human programming approach, LLMs directly generate the output in the target language based on the input query without verifying the correctness of the output. This approach has no safeguards against incorrect code, where even state-of-the-art LLMs are prone to syntactic errors, semantic mistakes, and possible model hallucinations (Song et al., 2023; Zhang et al., 2025b). Second, to fully understand the syntax of the target language, LLMs require a large dataset for training, which does not exist for low-resource programming languages (LRPLs) and is often difficult to construct due to the scarcity and fragmentation of high-quality training data for the target language and limited community usage. As a result, LLMs tend to show inferior performance when trained on LRPLs (Orlanski et al., 2023). Finally, to achieve the best results, finetuning the LLM on the code generation task is generally needed but this is a time-consuming process with high hardware requirements, especially for very large models.

Prior works attempt to tackle these problems by proposing advanced training techniques (Gong et al., 2022; Zhang et al., 2025a) or pipelines involving program fixing modules based on compiler feedback (Zhang et al., 2023; Wang et al., 2022). However, these methods only target one of the challenges and a unified pipeline that can effectively solve all three challenges is still missing.

In this paper, we focus on addressing these chal-

lenges to improve the quality of LLM-generated programs for domain-specific languages (DSLs). While general-purpose programming languages have vast datasets and well-established tooling, DSLs are often low-resource and lack large-scale datasets, which makes training LLMs for reliable code generation particularly challenging. Specifically, our goal is to eliminate statically detected errors, such as syntax and type errors, in LLM-generated programs, which are easy to detect but remain challenging for existing LLMs to avoid (Xia et al., 2023).

Recent studies (Kusama et al., 2025; Sheng and Xu, 2025) have showed the potential of small language models (SLMs) for code-related tasks, and the task of fixing syntactic errors could be very suitable for SLMs because the model only needs to be able to make corrections, not generate the code from scratch. Meanwhile, Reinforcement learning (RL) approaches (Le et al., 2022; Liu et al., 2023; Jha et al., 2024) are widely employed in training LLMs to generate high-quality programs because they support more flexible training targets compared to supervised finetuning (SFT), but also bring higher training costs compared to SFT. However, using RL techniques to train SLMs would be much cheaper yet remaining effective.

Based on this finding, we propose SLMFix, a novel code generation pipeline that can be applied to any existing LLMs pretrained on general code generation tasks without finetuning on the model, and performs well even for DSLs with limited training data. We achieve this by incorporating an SLM with 500M parameters finetuned with RL for error-fixing in the target language.

To ensure that the output of the SLM fixes the errors while aligning to the original prompt, we use a reward which combines a static validator, which accounts for both syntactic and other statically checkable correctness properties, and a semantic scorer that evaluates the functional correctness. The static validation can be performed by a variety of static tools, such as parsers, type-checkers, linters, and even symbolic interpreters, but semantic evaluation is much harder. Existing approaches (Zhang et al., 2023; Le et al., 2022; Orlanski et al., 2023) evaluate functional correctness of the generated program using sets of test cases but constructing such test suites requires significant effort and is necessarily far from complete since the range of valid programs and input data values for those programs is nearly unbounded. Instead, we build on

prior works (Liang et al., 2025; Song et al., 2024) showing that parsing the generated program and the corresponding ground-truth program into abstract syntax trees (ASTs) and comparing the similarities of the ASTs can be an effective method of checking functional correctness without the use of a test suite. This approach is especially well-suited to DSLs because the task-specific and well-structured nature of many DSLs means that they have a limited number of programs that achieve the same functionality. With careful analysis (outlined in Appendix B), we found that AST similarity is able to achieve more than 75% accuracy in comparison to testing.

To address the issue of data scarcity, we construct the training dataset for the finetuning by first manually crafting a very small dataset containing a matching set of natural language query and ground truth program that have a good coverage of the target language. We then employ LLMs to generate corresponding (potentially incorrect) programs as examples for error-fixing, which can significantly increase the amount of training data available.

We evaluate our proposed approach on four example DSLs: Ansible, Bash, SQL, and Lean. While Bash and SQL are high-resource languages with large scale training data, Ansible and Lean, which are popular tools for IT automation and theorem proving, remain low-resource languages due to limited dataset availability. To verify the effectiveness of our method on Ansible, we constructed an Ansible dataset consisting of Ansible playbooks scraped from open-source Ansible repositories. With comprehensive experiments on multiple LLMs and baselines, we show that SLMFix is able to bring substantial improvements to the base models across multiple DSLs while being very efficient, increasing the pass rate of static checker over 40% for LRPLs and fixing more than 50% of the static errors for high-resource DSLs, validating the effectiveness of our approach.

In summary, our contributions are as follows:

- We propose SLMFix, a novel code generation pipeline for DSLs that leverages a pre-trained LLM for initial code generation and an SLM fine-tuned using RL for error-fixing, and show it can significantly improve the quality of LLM-generated programs with low latency across multiple DSLs, including both high-resource and low-resource languages, which demonstrates the benefits of using SLMs for error fixing.

- We demonstrated the effectiveness of using semantic similarity based on ASTs to evaluate the functional correctness of LLM-generated programs in DSLs that can work as an alternative for a well-designed test suite.
- We constructed a new dataset for Ansible code generation, scraped from open-source repositories, that can be utilized for Ansible program generation training and evaluation.

2 Related Work

2.1 Reinforcement Learning for Code Gen

Reinforcement Learning (RL) has seen wide adoption in modern LLM training because it supports dynamic measurements of model performance during training and more flexible training targets compared to SFT with an objective of next token prediction. For code generation, RL methods are able to consider the syntactic and functional correctness by leveraging compilers and test suites to evaluate the generated code beyond token matching, making them more suitable for training than SFT. CodeRL (Le et al., 2022) proposed training a separate LLM to predict the functional correctness of generated programs using unit test results and the reference program. Subsequent works have improved this approach by introducing more fine-grained reward functions and training schemes (Shojaee et al., 2023; Dou et al., 2024; Liu et al., 2023; Jha et al., 2024; Ye et al., 2025).

However, RL approaches generally require a test suite to verify the functional correctness of the code, which is expensive to construct and error-prone. To address this issue, recent works (Wang et al., 2022; Jain et al., 2023) also proposed new training pipelines that learn from compiler feedback to generate more compilable codes without requiring test cases.

2.2 Automated Error Fixing

Automated error fixing leverages LLMs to diagnose and fix errors in code and is commonly employed in the code generation pipelines to improve the quality of LLM-generated code. Prior works have explored training-free methods (Ngassom et al., 2024; Shi et al., 2024), supervised finetuning (Zhang et al., 2023; Liu et al., 2024), and RL-based methods (Gupta et al., 2019; Zhao et al., 2024) for automated error fixing, showing that error fixing can significantly improve the quality of generated

programs. However, most of these approaches require large scale training data, which limits their effectiveness in LRPLs.

2.3 Small Language Models

Small Language Models (SLMs) have received lots of attention recently from the AI community as their smaller model size makes training and inference more efficient. Previous works (Sheng and Xu, 2025; Kusama et al., 2025; Koutchme et al., 2024) have explored training SLMs for code generation tasks and found that SLMs with 3B parameters can achieve comparable performance or even outperform state-of-the-art LLMs with over 10B parameters with proper training methods. Kusama et al. (2025) also attempted to employ SLMs for error fixing, but the authors only focused on high-resource languages like Python and Java which SLMs are already good at, while their capabilities on low-resource languages remain unexplored.

3 Methodology

Our proposed training and inference pipelines are summarized in Figure 1. We train a Small Language Model (SLM) with Group Relative Policy Optimization (GRPO) (Shao et al., 2024) to fix syntactic errors given the LLM-generated programs in a specific programming language and the feedback from a static validator of the language. During inference, our framework uses a frozen LLM pre-trained for general code generation to generate an initial version of the program given the natural language query, which is then passed into the static validator to detect errors such as syntax and type errors. If there are no errors the program is output and otherwise the program, the original prompt, and the error messages from the validator are provided to the SLM which generates a revised program as the final output.

In the rest of this section we discuss our training method for the SLM (Section 3.1), the validator we employed in our training and evaluation (Section 3.2), our approach to developing the semantic similarity metrics used in training (Section 3.3), and finally outline our process for generating a large amount of training data from a small initial dataset (Section 3.4).

3.1 Training

Supervised finetuning with next token prediction as training objective does not reflect the focus on

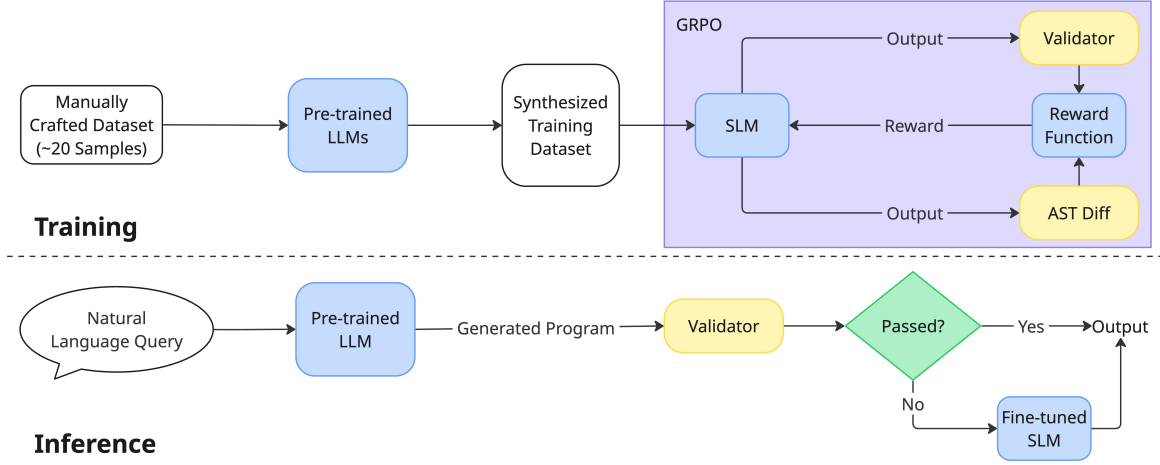


Figure 1: The overview of the proposed training and inference pipelines.

semantic correctness for code-related tasks that do not require token for token equivalence. Therefore, we use reinforcement learning with reward signals computed from the feedback of a static validator, which checks properties like syntax and types, and the score given by a semantic similarity metric, which compares the generated program with the ground truth to estimate the semantic correctness of the program. The static validator generates a boolean signal indicating if the given program contains errors, while the semantic metric will output a score between 0 and 1 representing how well the program aligns with the input query. The reward of the program is then a weighted average between these two signals, and the model parameters will be updated accordingly with Group Relative Policy Optimization (GRPO).

In order to balance between static and semantic correctness, we weight the scores from the validator and semantic metric in the reward function adaptively based on the percent of generated programs, in the current batch, that pass the validator. With this, the training focuses on improving static validity when most fixed programs still contain errors, and starts to focus on semantic correctness when fixed programs have a high pass rate on the validator. More formally, given a batch of programs $p_1, \dots, p_n$, the reward given to program p_i would be

$$r_i = (1 - r_{pass}) \cdot \mathbb{1}[f_s(p_i) = true] + r_{pass} \cdot f_f(p_i)$$

where $f_s(p) \in \{true, false\}$ is the response of the static validator, $f_f(p) \in [0, 1]$ evaluates the semantic similarity between the program and the

ground-truth, and $r_{pass} = \frac{1}{n} \sum \mathbb{1}[f_s(p_i) = true]$ is the pass rate of the current batch. r_{pass} is floored at 0.5 to avoid overemphasizing syntactic correctness.

3.2 Validator Design

For validating Ansible playbooks, we use Astroigator (Councilman et al., 2025) which is a symbolic interpreter for Ansible. This system includes a parser for (a subset of) Ansible; performs type-checking of the Ansible program, including ensuring variables are well-defined and that module argument are well typed; and ensures that arguments to module are appropriate (i.e., ensures that required arguments or mutually-exclusive arguments are provided appropriately). Additionally, it can perform symbolic interpretation of Ansible programs which simulates the behavior of the program on all different possible inputs but in this work, we only use the interpreter for static checking but the interpreter could also be used to run tests or to verify the behavior of the program matches some other reference, such as a formal specification (Councilman et al., 2025).

To validate Bash scripts, we use ShellCheck¹ to identify syntactic errors. ShellCheck will also report checkstyle and linting errors, but we only consider syntax errors in this work. For SQL, we reconstructed the database according to the schema provided by the dataset and executed the generated queries to detect syntax errors. For Lean, we used the builtin parser to parse the program and find potential errors.

¹<https://github.com/koalaman/shellcheck>

3.3 Semantic Similarity Metric Design

Domain-specific languages are typically task-specific, well-structured, and highly abstract languages. Therefore, commands and constructs in DSLs are generally high-level and tailored to a specific domain, which greatly limits the number of different ways to complete a specific task. From this observation, we propose to measure how well a generated program aligns with the input query by comparing the abstract syntax trees (ASTs) of the generated program and a reference ground-truth program. ASTs are the result of parsing the target language program and often abstract some syntactic details, meaning two syntactically different programs may have the same or very similar ASTs. The following subsections describe the language-specific AST-based semantic similarity metric we design for each language in our evaluation.

3.3.1 Ansible

Ansible programs (called playbooks), which we focus on generating, are composed of a sequence of *plays* that define the operations to perform. Each play consists of a sequence of *tasks* that define the atomic operations in the play, as well as other configurations that specify how the tasks should be executed.

Motivated by Pujar et al. (2023), we decompose each playbook into a sequence of plays and divide each play into a list of tasks, and parse the arguments to each task into a dictionary. For each task in the ground-truth, we find the corresponding task in the generated playbook and check if each key in the task dictionary appears in the corresponding task dictionary of the generated task and if the same value is assigned to the key. The final score assigned to a particular task is

$$score = \frac{\sum_{(k,v) \in \hat{y}} \mathbb{1}[(k,v) \in y]}{|\hat{y}|}$$

where $\hat{y}$ is the task dictionary of ground-truth and y is the task dictionary of the generated playbook. We then obtain the final score by averaging over all tasks in the play and all plays in the playbook.

3.3.2 Bash

Since there are no existing datasets for multi-line Bash script generation, we focus on single-line Bash scripts which can contain parentheses and connectors. We use Bashlex² to parse a Bash script

into *atomic commands*, where each atomic command executes a single bash command without any connectors. Similar to Ansible, we then parse each atomic command into a dictionary, from the flags and value passed to a command, and check if each flag in the ground-truth appears in the output and is associated with the same value. We obtain the overall score for a Bash script by taking the average over all of the atomic commands.

3.3.3 SQL

For SQL, we use SQLGlot³ to normalize a query and parse it into an AST. To improve the accuracy of the AST-based comparison, we remove all column aliases and replace table aliases with their original names. The semantic similarity score is then computed as the reciprocal of the tree edit distance between the AST of the generated program and the AST of the ground-truth.

3.3.4 Lean

To compare generated Lean programs against ground-truth, we examine from three aspects: overall token overlap, declaration overlap, and tactic overlap, which capture the most important parts of a Lean program. We calculate the matching in each aspect with F-1 score and compute the semantic similarity with the average of these scores.

3.4 Training Data Synthesis

To construct training data for our reinforcement learning, we use a multi-model generation strategy that leverages the diversity of outputs from different large language models to improve robustness and reduce overfitting. From each language-specific dataset, we manually constructed a subset of 20 diverse natural language queries along with corresponding ground truth code implementations. With this small set of curated examples, we generate 10 sample programs for each example from five strong code LLMs: StarCoder-2 7B (Lozhkov et al., 2024), DeepSeek-Coder 6.7B (Guo et al., 2024), Qwen2.5-Coder 7B (Hui et al., 2024), Granite 8B Code (Mishra et al., 2024), and LLaMA-3.1 8B (LlamaTeam and AI@Meta, 2024).

This multi-model generation process creates an expansive training dataset 50 times the size of original dataset consisting of natural language queries paired with both LLM-generated code (of varying quality) and corresponding ground truth implementations. The resulting dataset provides the varied

²<https://github.com/idank/bashlex>

³<https://github.com/tobymao/sqlglot>

examples necessary for training a strong and robust error-fixing model.

4 Ansible Dataset Construction

Since no existing dataset is publicly available for Ansible, we constructed an Ansible dataset through an end-to-end collection and parsing pipeline to gather real-world Ansible code. The data collection process began with the development of a custom web scraper built using Selenium to systematically crawl Ansible Galaxy⁴, the primary community hub for Ansible that contains Ansible repositories with open-source licenses, to identify and download all publicly available repositories listed on the platform.

Following the repository collection phase, the downloaded content underwent structured parsing using the Ansible Content Parser⁵. This specialized parser analyzes Ansible files to extract valid, parsable playbooks from the collected repositories and remove sensitive information from the repositories. The Ansible Content Parser examines the structure of Ansible content including playbooks, roles, and tasks, identifying syntactically correct playbooks while filtering out malformed or unparsable content.

A critical component of the parsing process involved variable and role infilling; these steps produce stand-alone executable programs rather than programs that rely on other files to resolve variables and imported roles. For variable infilling, we used the parser to identify variable references within playbooks and populated them with appropriate values based on variable definitions found in defaults, vars files, inventory configurations, and other sources following Ansible’s variable precedence rules. For role infilling, we collected all roles that are defined in the scope of the playbook and replace the roles by their corresponding definitions in the playbook.

The web scraping and parsing pipeline successfully yielded a corpus of 33,558 syntactically valid Ansible playbooks but these playbooks still lacked accompanying natural language descriptions that are necessary for training natural language-to-code generation tasks. We employed GPT-4o mini (OpenAI et al., 2024) to generate natural language descriptions for the collected playbook. To ensure the correctness of LLM labeled data, we conducted

manual inspection on the generated descriptions. The details of the dataset and example inputs/outputs can be found in Appendix A.

5 Evaluation

5.1 Experimental Setup

Target Programming Languages. We chose Ansible, Bash, SQL, and Lean as example DSLs for our evaluation for a diversity of languages, including high- and low-resource languages. Bash and SQL are high-resource DSLs with abundant training data, while Ansible and Lean are examples of low-resource programming languages despite being widely used in many fields.

Datasets. For Ansible code generation, we utilized the dataset we constructed, as described in Section 4. For Bash we used the NL2Bash dataset (Lin et al., 2018), a corpus that covers over 100 commonly used Bash utilities with diverse commands. For SQL command generation, we leveraged the Spider dataset (Yu et al., 2018), a large-scale human-labeled corpus designed specifically for complex and cross-domain semantic parsing and text-to-SQL tasks. The natural language queries were augmented with their corresponding database schemas to provides models with the necessary context to understand the database structure and generate correct SQL queries that reference valid table and column names. For Lean, we employed NuminaMath (Wang et al., 2025) dataset, which consists of mathematical competition problems formalized in Lean 4. For all datasets, we removed the samples that cannot pass our validator.

Baselines. To compare the performance of our proposed pipeline with prior works, we selected a variety of LLMs, including GPT-5.4 mini (?), Qwen3-Coder 30B (Team, 2025), Qwen-2.5-Coder 7B (Hui et al., 2024), DeepSeek-Coder 6.7B (Guo et al., 2024), and Granite-3.3 8B (Granite Team, 2024) models as the base LLMs and compared our proposed approach against the following types of baselines in our experiments:

- Pre-trained LLMs: directly generate the program given the input query;
- Supervised fine-tuning (SFT): finetune the base LLM on target language and employ fine-tuned LLMs for direct code generation;
- In-context learning (ICL): provide several examples of the target language to the base LLM

⁴<https://galaxy.ansible.com/>

⁵<https://github.com/ansible/ansible-content-parser>

	Ansible			Bash			SQL			Lean		
	CodeBERTScore	Pass Rate	AST Diff	CodeBERTScore	Pass Rate	AST Diff	CodeBERTScore	Pass Rate	AST Diff	CodeBERTScore	Pass Rate	AST Diff
GPT-5.4 mini												
Base	0.8010	25.00%	0.3240	0.8688	99.66%	0.5062	0.8905	99.32%	0.5383	0.7535	84.42%	0.1189
ICL	0.8421	71.31%	0.3573	0.8777	99.66%	0.5408	<u>0.9250</u>	99.32%	0.5909	0.7704	90.00%	0.1287
Self-correction	0.8015	36.74%	0.2839	0.8688	100.00%	0.5075	0.8707	99.90%	0.5398	0.7552	93.85%	0.1220
Self-Edit	0.7508	59.23%	0.2064	0.7525	100.00%	0.5054	0.8373	99.61%	0.5380	0.7296	85.19%	0.1088
SLMFix(Ours)	0.8321	<u>92.95%</u>	0.3054	0.8444	99.83%	0.5045	0.8855	99.71%	0.5383	0.7344	88.85%	0.1170
Qwen3-Coder 30B A3B												
Base	0.8121	44.63%	0.2961	0.869	99.66%	0.4883	0.8837	99.61%	0.5929	0.7222	39.23%	0.0778
ICL	0.8394	61.74%	<u>0.3346</u>	<u>0.8783</u>	99.49%	0.5136	0.9211	98.74%	0.6267	<u>0.7795</u>	46.73%	0.1010
Self-correction	0.8119	53.36%	0.3086	0.8692	99.83%	0.4883	0.8838	99.90%	0.5933	0.7341	44.62%	0.0803
Self-Edit	0.7452	72.32%	0.2706	0.7540	100.00%	0.4883	0.8388	99.61%	0.5929	0.7117	46.54%	0.0548
SLMFix(Ours)	<u>0.8395</u>	<u>96.48%</u>	0.3240	0.8446	99.83%	0.4883	0.8866	99.71%	0.5929	0.6562	<u>75.19%</u>	0.0534
Qwen-2.5-Coder 7B												
Base	0.8198	59.56%	0.3097	0.8692	98.46%	0.4948	0.9082	95.74%	0.5650	0.7320	15.96%	0.0881
SFT	0.8392	73.83%	0.3704	0.8923	99.32%	0.5566	0.9531	94.29%	0.6473	<u>0.7878</u>	7.31%	0.0988
ICL	0.8385	65.94%	0.3455	0.8751	99.83%	0.5152	0.9418	96.71%	0.6318	0.7734	54.23%	<u>0.1001</u>
Self-correction	0.8205	60.74%	0.3136	0.8693	98.63%	0.4961	0.9072	97.20%	0.5657	0.7340	16.54%	0.0885
Self-Edit	0.7399	75.84%	0.2958	0.7502	<u>99.83%</u>	0.4987	0.8363	96.52%	0.5648	0.7253	23.27%	0.0503
SLMFix(Ours)	<u>0.8403</u>	98.15%	0.3420	0.8451	<u>99.83%</u>	0.4982	0.8871	<u>97.87%</u>	0.5683	0.6505	<u>72.12%</u>	0.0620
DeepSeek-Coder 6.7B												
Base	0.8203	59.40%	0.3319	0.8511	100%	0.4359	0.8780	98.65%	0.4609	0.7292	9.62%	0.0977
SFT	0.8087	47.54%	0.3081	<u>0.8794</u>	97.95%	<u>0.5240</u>	<u>0.9189</u>	94.78%	<u>0.5753</u>	<u>0.7552</u>	21.35%	0.0890
ICL	0.8309	61.91%	<u>0.3530</u>	0.8603	99.49%	0.4652	0.8965	97.49%	0.5145	0.7469	<u>51.73%</u>	0.0100
Self-correction	0.8178	62.75%	0.3336	0.8502	100%	0.4359	0.8778	<u>99.81%</u>	0.4619	0.7331	10.00%	<u>0.1007</u>
Self-Edit	0.7395	77.52%	0.2989	0.7541	100%	0.4359	0.8376	98.94%	0.4607	0.7283	16.35%	0.0562
SLMFix(Ours)	<u>0.8370</u>	<u>97.65%</u>	0.3418	0.8420	100%	0.4359	0.8858	99.32%	0.4639	0.6636	42.31%	0.0686
Granite-3.3 8B												
Base	0.8147	45.30%	0.2891	0.8412	98.63%	0.4143	0.8759	90.23%	0.3929	0.7585	33.65%	0.1101
SFT	0.8298	82.89%	0.2945	0.8342	93.16%	0.3850	<u>0.9074</u>	64.89%	0.3820	0.7888	10.77%	0.1012
ICL	0.8331	54.53%	<u>0.3262</u>	<u>0.8546</u>	98.97%	<u>0.4520</u>	0.8831	91.39%	<u>0.4154</u>	0.7859	46.15%	<u>0.1164</u>
Self-correction	0.8134	51.68%	0.3014	0.8402	98.97%	0.4143	0.8715	<u>95.65%</u>	0.3987	0.7597	37.88%	0.1110
Self-Edit	0.7405	66.28%	0.2356	0.7516	99.32%	0.4169	0.8331	91.59%	0.3928	0.7280	39.04%	0.0732
SLMFix(Ours)	<u>0.8403</u>	<u>96.31%</u>	0.3109	0.8407	<u>99.49%</u>	0.4167	0.8826	94.68%	0.4038	0.6520	<u>73.85%</u>	0.0777

Table 1: Evaluation results on Ansible, Bash, SQL, and Lean code generation. The best performance in each column is marked in **bold**, while the best performance in each group is marked in underline. SLMFix is able to achieve the highest pass rate on the validator across all languages and LLMs, achieving competitive performance and even outperforms the strongest baselines on semantic metrics.

for direct code generation;

- Self-correction: provide the base LLM with the error message from the validator and ask it to revise the original program;
- Automatic program repair methods: fine-tune the SLM for error-fixing with Self-Edit (Zhang et al., 2023), the example of automatic program repair methods we selected.

Due to consideration of finetuning cost, we only performed SFT on Qwen-2.5-Coder 7B, DeepSeek-Coder 6.7B, and Granite-3.3 8B. For our error-fixing model, we used Qwen-2.5 Coder 0.5B (Hui et al., 2024) which we fine-tune for the task. For in-context learning, we randomly selected five samples from the training dataset as the in-context examples.

Metrics. For all languages, we report CodeBERTScore (Zhou et al., 2023) to evaluate the semantic similarity between the generated program and the ground-truth. For each language, we also report the pass rate on the corresponding validator and the average score on the semantic similarity metric described in Section 3.3.

5.2 Implementation Details

For reinforcement learning approaches, we use verl (Sheng et al., 2024) and vLLM (Kwon et al., 2023) to perform training with 70 total epochs and batch size of 64 on the Qwen-2.5-Coder 0.5B model. We set the maximum input and output length to be 4096 tokens, and the learning rates for the model as 1e-6, respectively. For supervised finetuning, we employed LoRA (Hu et al., 2022) for parameter-efficient finetuning. We set the maximum number of steps to be 150 and batch size as 4, with learning rate of 1e-4 and weight decay of 0.05. At inference time, we use vLLM for efficient generation, setting temperature to 0 to avoid randomness. We conducted all experiments on two 40GB A40 GPUs and the experiments took roughly 100 GPU hours in total to complete.

5.3 Main Results

Table 1 presents the evaluation results of SLMFix and baselines on Ansible, Bash, SQL, and Lean code generation using difference LLMs as the base model. SLMFix is able to significantly improve the syntactic correctness of LLM-generated programs while remaining aligned with the user intent. For Ansible and Lean, which are low-resource languages, SLMFix is able to improve the pass rate of

static checking by an average of over 40%, which is higher than most baselines we compared against. For Bash and SQL, although the base LLMs are already able to eliminate most syntactic errors in the generated programs, SLMFix is still able to further improve the quality of the programs by correcting more than 50% of the static errors in the output. Results on CodeBERTScore and AST Diff also show that SLMFix is able to preserve the semantic meaning of the original program when fixing syntactic errors, verifying the effectiveness of our proposed pipeline.

5.4 Efficiency Analysis

To better understand the efficiency of SLMFix compared to the baseline, we also analyzed the latency of SLMFix in code generation, as shown in Table 2. The results show that SLMFix takes almost no additional time after initial generation to fix syntactic errors, faster than most baselines across different LLMs. This result verifies that SLMFix can be a very efficient code generation pipeline that can eliminate most syntactic errors.

	Base	SFT	ICL	Self-correction	Self-Edit	SLMFix (Ours)
Qwen-3-Coder 30B A3B	60.29	-	51.41	133.23	184.87	68.50
Qwen-2.5-Coder 7B	42.52	132.84	75.58	94.75	172.34	45.73
DeepSeek-Coder 6.7B	124.00	263.27	1009.67	370.35	255.20	126.72
Granite-3.3 8B	278.67	179.31	429.52	587.36	406.83	282.06

Table 2: Latency comparison of baselines and SLMFix on Ansible code generation. Latency is measured as total generation time in seconds for the evaluation dataset of size 596. For program repair methods, the latency is calculated as sum of generation (Base) and fixing.

5.5 Ablation Studies

To better understand how different components of the training approach affect the overall performance of our proposed framework, we conducted a series of ablation studies on the components. To highlight the performance difference across various settings, we select Ansible as the target language and employ Qwen-2.5 0.5B (Yang et al., 2024) and LLaMA-3.2 1B (LlamaTeam and AI@Meta, 2024) as the base models to amplify the differences between design choice. Table 3 summarizes the results of the ablation study. We found that using only the static validator score or the semantic similarity metric would achieve excellent performance on the particular metric the model was trained on, but also cause the trained model to overfit to the training target while ignoring other metrics, resulting in serious performance degradation. In contrast,

combining both metrics achieves a better balance between static validation and functional correctness, helping the model to generate higher quality programs. In addition, we also compared with using the proposed RL method to directly train the SLM on the code generation task to verify if using SLM for error fixing is needed. We found that the reinforcement learning method we utilized does not work well on direct code generation task, possibly due to the limited model capability of SLMs. This result demonstrates the benefits of leveraging SLMs for program repair over direct generation.

	BLEU	CodeBERTScore	Pass Rate	AST Diff
RL for code generation	0.3856	0.8101	48.83%	0.2792
Qwen-2.5 0.5B				
SLMFix	<u>0.3913</u>	0.8274	94.30%	0.3455
Syntactic score only	0.3265	<u>0.8298</u>	<u>98.99%</u>	0.2062
Semantic score only	0.2553	0.7588	22.48%	<u>0.4047</u>
LLaMA-3.2 1B				
SLMFix	<u>0.4104</u>	0.8290	92.79%	0.3558
Syntactic score only	0.3612	<u>0.8365</u>	<u>99.50%</u>	0.2353
Semantic score only	0.2462	0.7430	19.63%	<u>0.3601</u>

Table 3: The results of ablation studies. The best performance in each column is marked in **bold**, while the best performance in each group is marked in underline. Our proposed training method can achieve an excellent balance between the syntactic and functional correctness compared to the ablated settings.

6 Conclusion

In this paper, we present SLMFix, a new pipeline for code generation that involves a pretrained LLM and an SLM tailored for error-fixing. We propose to leverage deep reinforcement learning techniques to train the small language model to fix identified errors in LLM-generated programs, using a fine-grained reward function that evaluates both syntactic and semantic correctness of the fixed programs. This approach is especially well suited to DSLs as they typically have well-structured syntax and high-level functions. We also verified the effectiveness of using similarity of abstract syntax trees as an estimate of functional correctness for DSLs to replace the need for a well-designed test suite, making training for code generation tasks much easier. With extensive experiments on Ansible, Bash, SQL, and Lean, we demonstrated that our proposed method is able to significantly improve the quality of LLM-generated program by reducing syntactic errors, outperforming similar program repair approaches and achieving comparable performance with LLM finetuned on the task.

Limitations

We acknowledge several limitations of our work. First, the semantic similarity metrics we designed are still not accurate enough to fully replace test suites. Although our analysis showed that semantic similarity measurements are suitable for DSLs based on their characteristics and show high correlation with other metrics including LLM judgment and pass rate on test suites, they are too strict and have low true positive rates. Second, we only conducted experiments on four DSLs, including two high-resource languages and two low-resource languages, which cannot completely validate the effectiveness of the proposed approach across all DSLs. Conducting experiments on other DSLs could help to strengthen our claims. Third, our method relies on an efficient syntactic checker for the target language. Although this is generally available for most languages, manually implementing such a checker might be needed for certain languages like Ansible, which limits the usability of our pipeline.

References

- Anthropic. 2024. [The Claude 3 Model Family: Opus, Sonnet, Haiku](#).
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. [Evaluating large language models trained on code](#). *Preprint*, arXiv:2107.03374.
- Aaron Councilman, David Fu, Aryan Gupta, Chengxiao Wang, David Grove, Yu-Xiong Wang, and Vikram Adve. 2025. [Towards formal verification of llm-generated code from natural language prompts](#). *Preprint*, arXiv:2507.13290.
- Shihan Dou, Yan Liu, Haoxiang Jia, Enyu Zhou, Limao Xiong, Junjie Shan, Caishuang Huang, Xiao Wang, Xiaoran Fan, Zhiheng Xi, Yuhao Zhou, Tao Ji, Rui Zheng, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. [StepCoder: Improving Code Generation with Reinforcement Learning from Compiler Feedback](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, pages 4571–4585. Association for Computational Linguistics.
- Google Gemini Team. 2025. [Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities](#). *Preprint*, arXiv:2507.06261.
- Zi Gong, Yinpeng Guo, Pingyi Zhou, Cuiyun Gao, Yasheng Wang, and Zenglin Xu. 2022. [MultiCoder: Multi-Programming-Lingual Pre-Training for Low-Resource Code Completion](#). *CoRR*, abs/2212.09666. ArXiv: 2212.09666.
- IBM Granite Team. 2024. [Granite 3.0 language models](#).
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. [DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence](#). *CoRR*, abs/2401.14196. ArXiv: 2401.14196.
- Rahul Gupta, Aditya Kanade, and Shirish K. Shevade. 2019. [Deep Reinforcement Learning for Syntactic Error Repair in Student Programs](#). In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*, pages 930–937. AAAI Press.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-Rank Adaptation of Large Language Models](#). In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, An Yang, Rui Men, Fei Huang, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. [Qwen2.5-Coder Technical Report](#). *CoRR*, abs/2409.12186. ArXiv: 2409.12186.
- Abhinav Jain, Chima Adiole, Swarat Chaudhuri, Thomas W. Reps, and Chris Jermaine. 2023. [Tuning Models of Code with Compiler-Generated Reinforcement Learning Feedback](#). *CoRR*, abs/2305.18341. ArXiv: 2305.18341.
- Piyush Jha, Prithwish Jana, Arnav Arora, and Vijay Ganesh. 2024. [RLSF: Reinforcement Learning via Symbolic Feedback](#). *CoRR*, abs/2405.16661. ArXiv: 2405.16661.
- Charles Koutchme, Nicola Dainese, Sami Sarsa, Juho Leinonen, Arto Hellas, and Paul Denny. 2024. [Benchmarking Educational Program Repair](#). *CoRR*, abs/2405.05347. ArXiv: 2405.05347.
- Kazuki Kusama, Honglin Shu, Masanari Kondo, and Yasutaka Kamei. 2025. [How Small is Enough? Empirical Evidence of Quantized Small Language Models for Automated Program Repair](#). *CoRR*, abs/2508.16499. ArXiv: 2508.16499.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E.

862	<i>Machine Learning Research</i> , pages 26619–26645.	
863	PMLR.	
864	Saurabh Pujar, Luca Buratti, Xiaojie Guo, Nico-	
865	las Dupuis, Burn L. Lewis, Sahil Suneja, Atin	
866	Sood, Ganesh Nalawade, Matthew Jones, Alessan-	
867	dro Morari, and Ruchir Puri. 2023. Invited: Auto-	
868	mated Code generation for Information Technology	
869	Tasks in YAML through Large Language Models.	
870	In <i>60th ACM/IEEE Design Automation Conference,</i>	
871	<i>DAC 2023, San Francisco, CA, USA, July 9-13, 2023,</i>	
872	pages 1–4. IEEE.	
873	Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu,	
874	Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu,	
875	and Daya Guo. 2024. DeepSeekMath: Pushing the	
876	Limits of Mathematical Reasoning in Open Lan-	
877	guage Models. <i>CoRR</i> , abs/2402.03300. ArXiv:	
878	2402.03300.	
879	Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin	
880	Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin	
881	Lin, and Chuan Wu. 2024. Hybridflow: A flexible	
882	and efficient rlhf framework. <i>arXiv preprint arXiv:</i>	
883	<i>2409.19256.</i>	
884	Lei Sheng and Shuai-Shuai Xu. 2025. SLM-SQL: An	
885	Exploration of Small Language Models for Text-to-	
886	SQL. <i>CoRR</i> , abs/2507.22478. ArXiv: 2507.22478.	
887	Yuling Shi, Songsong Wang, Chengcheng Wan, and Xi-	
888	aodong Gu. 2024. From Code to Correctness: Clos-	
889	ing the Last Mile of Code Generation with Hierar-	
890	chical Debugging. <i>CoRR</i> , abs/2410.01215. ArXiv:	
891	2410.01215.	
892	Parshin Shojaee, Aneesh Jain, Sindhu Tipirneni, and	
893	Chandan K. Reddy. 2023. Execution-based Code	
894	Generation using Deep Reinforcement Learning.	
895	<i>Trans. Mach. Learn. Res.</i> , 2023.	
896	Da Song, Zijie Zhou, and Zhijie Wang. 2023. An Empir-	
897	ical Study of Code Generation Errors made by Large	
898	Language Models.	
899	Yewei Song, Cedric Lothritz, Xunzhu Tang,	
900	Tegawendé F. Bissyandé, and Jacques Klein.	
901	2024. Revisiting Code Similarity Evaluation with	
902	Abstract Syntax Tree Edit Distance. In <i>Proceedings</i>	
903	<i>of the 62nd Annual Meeting of the Association for</i>	
904	<i>Computational Linguistics, ACL 2024 - Short Papers,</i>	
905	<i>Bangkok, Thailand, August 11-16, 2024,</i> pages	
906	38–46. Association for Computational Linguistics.	
907	Qwen Team. 2025. Qwen3 Technical Report. <i>CoRR</i> ,	
908	abs/2505.09388. ArXiv: 2505.09388.	
909	Richard J. Waldinger and Zohar Manna. 1975. Knowl-	
910	edge and Reasoning in Program Synthesis. In <i>Ad-</i>	
911	<i>vanace Papers of the Fourth International Joint Con-</i>	
912	<i>ference on Artificial Intelligence, Tbilisi, Georgia,</i>	
913	<i>USSR, September 3-8, 1975,</i> pages 288–295.	
914	Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas	
915	Baksys, Junqi Liu, Marco Dos Santos, Flood Sung,	
916	Marina Vinyes, Zhenzhe Ying, Zekai Zhu, Jianqiao	
	Lu, Hugues de Saxcé, Bolton Bailey, Chendong Song,	917
	Chenjun Xiao, Dehao Zhang, Ebony Zhang, Fred-	918
	erick Pu, Han Zhu, and 21 others. 2025. Kimina-	919
	Prover Preview: Towards Large Formal Reason-	920
	ing Models with Reinforcement Learning. <i>CoRR</i> ,	921
	abs/2504.11354. ArXiv: 2504.11354.	922
	Xin Wang, Yasheng Wang, Yao Wan, Fei Mi, Yitong Li,	923
	Pingyi Zhou, Jin Liu, Hao Wu, Xin Jiang, and Qun	924
	Liu. 2022. Compilable Neural Code Generation with	925
	Compiler Feedback. In <i>Findings of the Association</i>	926
	<i>for Computational Linguistics: ACL 2022, Dublin,</i>	927
	<i>Ireland, May 22-27, 2022,</i> pages 9–19. Association	928
	for Computational Linguistics.	929
	Chunqiu Steven Xia, Yuxiang Wei, and Lingming	930
	Zhang. 2023. Automated Program Repair in the	931
	Era of Large Pre-trained Language Models. In <i>45th</i>	932
	<i>IEEE/ACM International Conference on Software En-</i>	933
	<i>gineering, ICSE 2023, Melbourne, Australia, May</i>	934
	<i>14-20, 2023,</i> pages 1482–1494. IEEE.	935
	An Yang, Baosong Yang, Beichen Zhang, Binyuan	936
	Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayi-	937
	heng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian	938
	Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Ji-	939
	axi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and	940
	22 others. 2024. Qwen2.5 Technical Report. <i>CoRR</i> ,	941
	abs/2412.15115. ArXiv: 2412.15115.	942
	Yufan Ye, Ting Zhang, Wenbin Jiang, and Hua Huang.	943
	2025. Process-Supervised Reinforcement Learning	944
	for Code Generation. <i>CoRR</i> , abs/2502.01715. ArXiv:	945
	2502.01715.	946
	Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga,	947
	Dongxu Wang, Zifan Li, James Ma, Irene Li,	948
	Qingning Yao, Shanelle Roman, Zilin Zhang, and	949
	Dragomir R. Radev. 2018. Spider: A Large-Scale	950
	Human-Labeled Dataset for Complex and Cross-	951
	Domain Semantic Parsing and Text-to-SQL Task.	952
	In <i>Proceedings of the 2018 Conference on Empir-</i>	953
	<i>ical Methods in Natural Language Processing, Brus-</i>	954
	<i>sels, Belgium, October 31 - November 4, 2018,</i> pages	955
	3911–3921. Association for Computational Linguis-	956
	tics.	957
	Jipeng Zhang, Jianshu Zhang, Yuanzhe Li, Renjie Pi,	958
	Rui Pan, Runtao Liu, Ziqiang Zheng, and Tong	959
	Zhang. 2025a. Bridge-Coder: Transferring Model	960
	Capabilities from High-Resource to Low-Resource	961
	Programming Language. In <i>Findings of the Asso-</i>	962
	<i>ciation for Computational Linguistics, ACL 2025,</i>	963
	<i>Vienna, Austria, July 27 - August 1, 2025,</i> pages	964
	10865–10882. Association for Computational Lin-	965
	guistics.	966
	Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023.	967
	Self-Edit: Fault-Aware Code Editor for Code Gen-	968
	eration. In <i>Proceedings of the 61st Annual Meet-</i>	969
	<i>ing of the Association for Computational Linguis-</i>	970
	<i>tics (Volume 1: Long Papers), ACL 2023, Toronto,</i>	971
	<i>Canada, July 9-14, 2023,</i> pages 769–787. Associa-	972
	tion for Computational Linguistics.	973

Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025b. [LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation](#). *Proc. ACM Softw. Eng.*, 2(ISSTA):481–503.

Yuze Zhao, Zhenya Huang, Yixiao Ma, Rui Li, Kai Zhang, Hao Jiang, Qi Liu, Linbo Zhu, and Yu Su. 2024. [RePair: Automated Program Repair with Process-based Feedback](#). In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Findings of ACL, pages 16415–16429. Association for Computational Linguistics.

Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic evaluation for text-to-sql with distilled test suite. In *The 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.

Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. [CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 13921–13937. Association for Computational Linguistics.

A Details of Ansible Dataset

Table 4 shows the statistics of the data collection process. We only keep the playbooks that can be successfully parsed and validated, without any undefined variables or duplicate contents.

Category	Count
Repositories Scraped from Old Ansible Galaxy	34057
Repositories Scraped from New Ansible Galaxy	3212
Total Repositories Scraped	37269
Total Playbooks Extracted	74497
Playbooks with Undefined Variables	5906
Playbooks with Duplicate Contents	35033
Playbooks Successfully Parsed	33558
Final Dataset Size	616

Table 4: Summary of Ansible repository and playbook collection statistics.

The following are some examples from the dataset. With manual inspections we found that the LLM-generated prompts align well with human-written prompts.

Example 1

Prompt: Generate an Ansible playbook that installs and configures a database on the localhost with root access.

Playbook:

```
---
- hosts: localhost
  remote_user: root
  tasks:
    - name: install postgresql
      yum: name=postgresql state=latest
      become: true
      become_method: sudo
```

1009

Example 2

Prompt: Generate an Ansible playbook that updates the apt package cache on Ubuntu hosts with elevated privileges.

Playbook:

```
---
- name: Prepare
  hosts: ubuntu
  become: true
  tasks:
    - name: Update apt packages
      apt:
        update_cache: true
```

1010

Example 3

Prompt: Generate an Ansible playbook that configures passwordless sudo access for the 'admin' group on macOS.

Playbook:

```
---
- name: Prepare
  hosts: macos
  become: true
  tasks:
    - name: Allow 'admin' group to have passwordless sudo
      ansible.builtin.lineinfile:
        dest: /etc/sudoers
        state: present
        regexp: ^%admin
        line: '%admin ALL=(ALL) NOPASSWD: ALL'
        validate: visudo -cf %s
```

1011

B Empirical Analysis of Semantic Similarity Metrics

To verify the effectiveness of our proposed semantic similarity metrics, we compared the scores given by semantic similarity with the testing results on Ansible and SQL, two languages with existing test suites available.

For SQL, we utilized the evaluation set of the Spider dataset (Yu et al., 2018) with test suites provided by Zhong et al. (2020). We randomly selected 100 outputs for each LLM we evaluated to run the test suites, and the comparison between execution match and semantic similarity is shown in Table 5. When considering having a semantic similarity of at least 0.8 as equivalent to the ground-truth, the metric is able to classify most outputs correctly with an accuracy of 76%, a true positive rate of 64.2% and a true negative rate of 98.8%, which validates its effectiveness in estimating functional correctness.

	Tests Failed	Tests Passed
$0.8 \leq \text{Sim} \leq 1$	2	212
$0.6 \leq \text{Sim} < 0.8$	0	0
$0.4 \leq \text{Sim} < 0.6$	15	6
$0.2 \leq \text{Sim} < 0.4$	24	35
$0 \leq \text{Sim} < 0.2$	129	77

Table 5: Comparison between execution match and semantic similarity for SQL. The overall accuracy of semantic similarity with respect to execution match is 76%.

For Ansible, we employed the benchmark suite constructed by Councilman et al. (2025), which contains 21 questions and 1,260 outputs, and the comparison between testing result and semantic similarity is shown in Table 6. When considering having a semantic similarity of at least 0.8 as equivalent to the ground-truth, the metric is able to classify most outputs correctly with an accuracy of 84.2% and a true negative rate of 94.6%. However, the true positive rate is only 38.4%, which indicates that the AST-based semantic similarity is too strict and fails to correctly classify Ansible programs that are functionally equivalent to the ground-truth but not semantically equivalent.

	Tests Failed	Tests Passed
$0.8 \leq \text{Sim} \leq 1$	56	89
$0.6 \leq \text{Sim} < 0.8$	89	89
$0.4 \leq \text{Sim} < 0.6$	95	6
$0.2 \leq \text{Sim} < 0.4$	170	10
$0 \leq \text{Sim} < 0.2$	618	38

Table 6: Comparison between testing result and semantic similarity for Ansible. The overall accuracy of semantic similarity with respect to testing is 84.21%.

C Prompt Templates

C.1 Code Generation Prompts

Ansible Generation Prompt

You are an expert in Ansible. The user will give you a task description and ask you to generate an Ansible playbook to complete the given task. You only need to output the content of the playbook. DO NOT use any shell commands (ansible.builtin.shell, ansible.builtin.command, etc.) in the playbook. Task: {task}
Answer: ““yaml

Bash Generation Prompt

You are an expert in Bash. The user will give you a task description and ask you to generate a bash command to complete the given task. You only need to output the content of the command. Task: {task}
Answer: ““bash

SQL Generation Prompt

You are an expert in SQL. The user will give you a task description and ask you to generate a SQL command to complete the given task. You only need to output the content of the command. Task: {task}
Answer: ““sql

Lean Generation Prompt

You are an expert in Lean 4 theorem proving and functional programming. The user will give you a problem description and a theorem in Lean 4 that solves the problem, and ask you to generate Lean 4 code (definitions, theorems, and proofs) to prove the theorem. You only need to output the Lean 4 code. Do NOT use ‘sorry’.

{task}

Answer: “lean

Bash Program Repair Prompt

You are an expert in Bash. You are asked to fix a possibly incorrect Bash command. You will be provided with the command to fix, the user input, and feedback from an interpreter that lists all syntactic errors in the command. Your goal is to fix the syntactic errors in the command (if any) while following the user’s instruction. You only need to output the content of the modified command.

User query: {query}

Original command:
{output}

Interpreter feedback:
{feedback}

Answer: “bash

C.2 Program Repair Prompts

Ansible Program Repair Prompt

You are an expert in Ansible. You are asked to fix a possibly incorrect Ansible playbook. You will be provided with the playbook to fix, the user input, and feedback from an interpreter that lists all syntactic errors in the playbook. Your goal is to fix the syntactic errors in the playbook (if any) while following the user’s instruction. You only need to output the content of the modified playbook.

User query: {query}

Original playbook:
{output}

Interpreter feedback:
{feedback}

Answer: “yaml

SQL Program Repair Prompt

You are an expert in SQL. You are asked to fix a possibly incorrect SQL command. You will be provided with the command to fix, the user input, and feedback from an interpreter that lists all syntactic errors in the command. Your goal is to fix the syntactic errors in the command (if any) while following the user’s instruction. You only need to output the content of the modified command.

User query: {query}

Original command:
{output}

Interpreter feedback:
{feedback}

Answer: “sql

Lean Program Repair Prompt

You are an expert in Lean 4 theorem proving and functional programming. You are asked to fix possibly incorrect Lean 4 code. You will be provided with the code to fix, the problem and theorem to prove, and feedback from the Lean type checker that lists all errors. Your goal is to fix the errors (if any) while following the user's instruction. Do NOT use 'sorry'. You only need to output the corrected Lean 4 code.

User query: {query}

Original code:
{output}

Lean checker feedback:
{feedback}

Answer: “lean

Bash In-context Learning Prompt

You are an expert in Bash. The user will give you a task description and ask you to generate a bash command to complete the given task. You only need to output the content of the command.

The following are some example input queries and corresponding Bash commands for your reference:

{examples}

Task: {task}

Answer: “bash

SQL In-context Learning Prompt

You are an expert in SQL. The user will give you a task description and ask you to generate a SQL command to complete the given task. You only need to output the content of the command.

The following are some example input queries and corresponding SQL commands for your reference:

{examples}

Task: {task}

Answer: “sql

C.3 In-context Learning Prompts

Ansible In-context Learning Prompt

You are an expert in Ansible. The user will give you a task description and ask you to generate an Ansible playbook to complete the given task. You only need to output the content of the playbook. DO NOT use any shell commands (ansible.builtin.shell, ansible.builtin.command, etc.) in the playbook. The following are some example input queries and corresponding Ansible playbooks for your reference:

{examples}

Task: {task}

Answer: “yaml

Lean In-context Learning Prompt

You are an expert in Lean 4 theorem proving and functional programming. The user will give you a problem description and a theorem in Lean 4 that solves the problem, and ask you to generate Lean 4 code (definitions, theorems, and proofs) to prove the theorem. You only need to output the Lean 4 code. Do NOT use 'sorry'.

The following are some example inputs and corresponding Lean 4 code for your reference:

{examples}

{task}

Answer: “lean

1062
1063

C.4 Ansible Dataset Query Generation Prompt

Natural Language Query Generation Prompt

You are an expert in Ansible. You are asked to write a user prompt for the given Ansible playbook that can be used to generate the playbook. Instead of explicitly describing the functionality of the playbook, the prompt should tell what the user wants to accomplish through the playbook. Write the prompt as short as you can, and start the prompt with: Generate an Ansible playbook that ...

1064